

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART VII

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: **Collaborative Workflow: Claude Code + GitHub + Jira**

Collaborative Workflow

Claude Code + GitHub

Instructor: Muhammad Arif Butt, PhD

Two Models of Collaborative Git Workflow

MODEL A — Fork & Pull Request

Contribute even without write access to the owner's repo.

1 Fork the repo — GITHUB

Copy the owner's repository into your own GitHub account.

2 Clone your fork — LOCAL

Bring your fork down onto your local machine.

3 Branch & build — LOCAL

Create a feature branch and do your work locally.

4 Push & open a PR — GITHUB

Push the branch to your fork, then open a Pull Request to the owner's repo.

MODEL B — Shared Repo (Direct Clone)

For trusted collaborators who have write access.

1 Clone the repo — LOCAL

With write access, clone the owner's repo directly.

2 Branch & build — LOCAL

Create a feature branch and do your work locally.

3 Push & open a PR — GITHUB

Push the branch to the owner's repo, then open a PR.

Both models converge below — the owner is the single merge point.

Owner / Maintainer — Review & Merge

Reviews the PR against the spec & plan, requests changes if needed, then merges into main. Collaborators cannot merge — the owner is the only gatekeeper of main.

Everyone Re-syncs main

After the merge, every collaborator updates their local main to match the remote — then branches off again for the next feature or fix. ↻ repeat.

Collaborator Generates PR

1 Clone the repo

```
git clone <repo-url>
cd <repo>
```

2 Sync latest main

```
git checkout main
git pull origin main
```

3 Create a branch

```
git checkout -b <branch>
# e.g. feature/.. or fix/..
```

4 Write spec + plan

```
.claude/specs/
<name>.spec.md
<name>.plan.md
```

5 Generate w/ Claude Code

```
claude
> build from spec + plan
# generates the code
```

6 Run tests & iterate

Run the test suite.

Fix via Claude Code.

Repeat until tests pass.

https://arifpucit.github.io/spec-code-generation-workflow/

7 Commit & push

```
git add -A
git commit -m "msg (#issue)"
git push -u origin <branch>
```

8 Open PR on GitHub

1. Click "New PR" button: add a title & description
2. Click "Compare PR" button: base=main & compare= <branch>
3. Wait for maintainer review / comments

✓ Owner-only step

Reviews & approves the PR

Merges it into main

Tags the release

(collaborators can't merge)

Step 8 CLI alternative: gh pr create --base main

Maintainer/Owner Accept PR & Gen Tag

1 Review PR on GitHub

Open the PR → "Files changed" tab
Confirm it touches only what it should
Spec + plan exist · code matches them
If anything's missing → request changes

2 Pull the branch

```
git checkout main
git pull origin main
git fetch origin
git checkout <branch>
```

3 Run the app to verify

```
git merge main
uv sync
uv run app/main.py
git push
```

4 Edit CLAUDE.md + README.md

```
git add -A
git commit -m "message"
git push
```

5 Approve & merge (UI)

Refresh PR → Review changes → Approve
"Merge pull request" → "Confirm merge"
Delete branch (optional)

6 Sync your local machine

```
git checkout main
git pull origin main
```

7 Create a tag

```
git tag -a v0.1.x -m "message"
git push origin v0.1.x
```

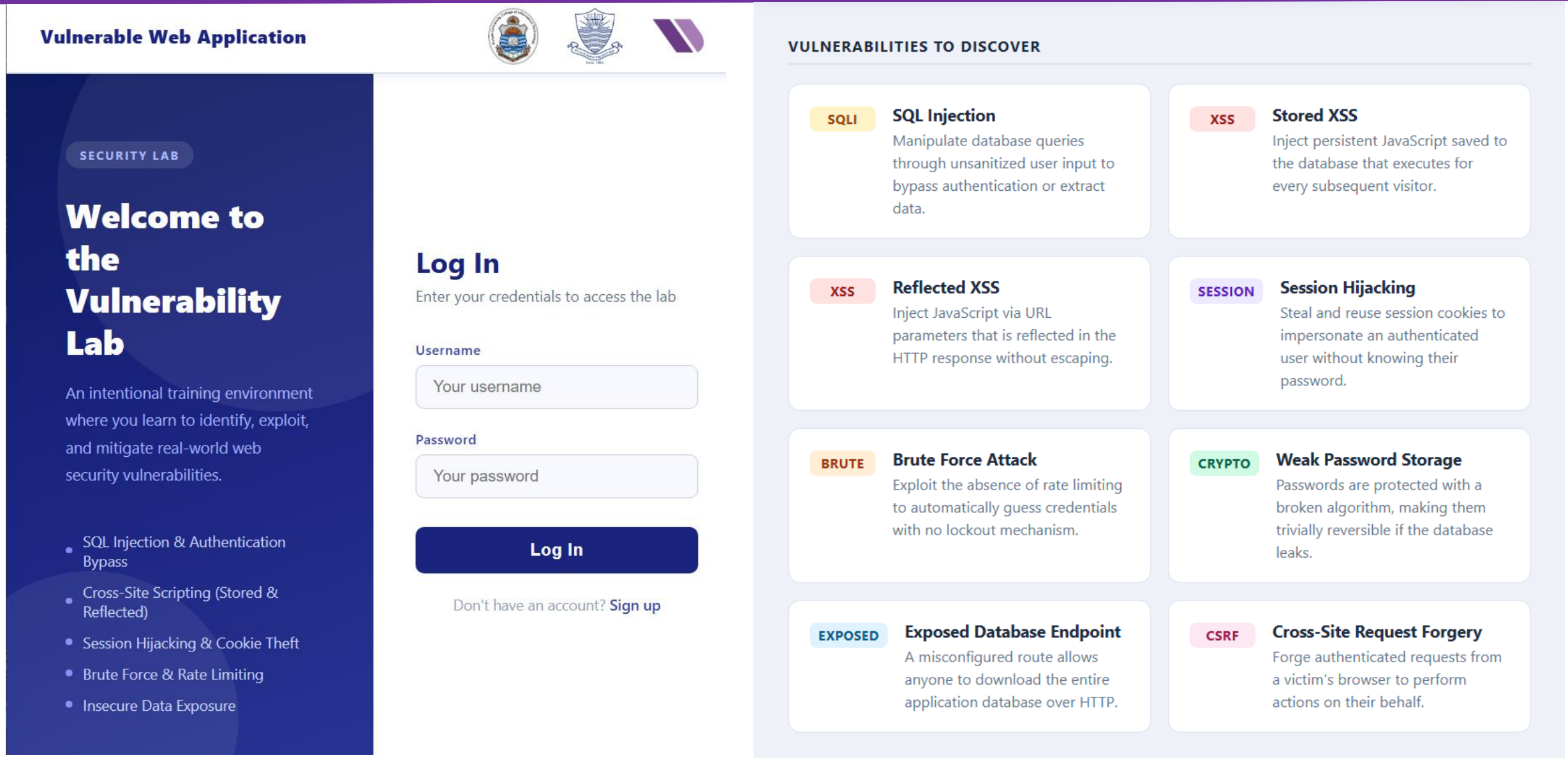
★ Publish a Release

Once ALL fixes are merged & tagged:
Draft a GitHub Release
from the latest tag
+ release notes

Assumes collaborators work on a **shared repo** (not forks); for a fork-based PR, replace step 2 with `gh pr checkout <#>`.

One tag per fix (`v0.1.x`) · publish a Release only after all fixes are merged · the `gh` CLI can run any GitHub step above.

Current Status of Vuln-Web-App Repo on GitHub



Add Feature: Password Strength Meter

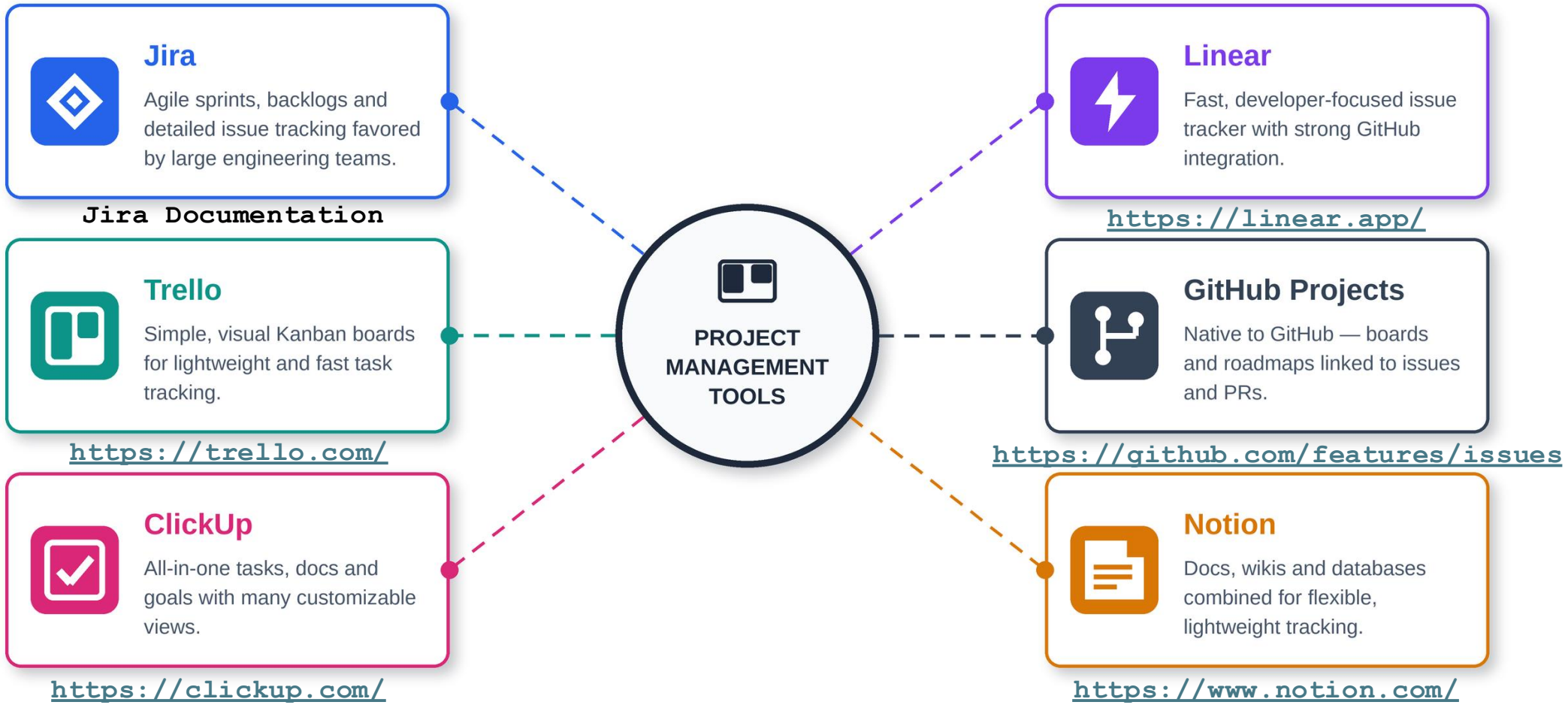


Collaborative Workflow

Claude Code + GitHub + Jira

Instructor: Muhammad Arif Butt, PhD

PM Tools for Collaborative s/w Dev



Jira Hand-out

Q/A



Thank you!